CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Data Structures & Algorithms: Placement Master Edition (Prescribed Textbook)

Subject Track: Data Structures & Algorithms (DSA)

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Authoritative campus placement textbook covering Data Structures and Algorithms from foundational asymptotic complexity to advanced dynamic programming, graphs, tries, DSU, and solved FAANG coding problems. Every chapter contains core concepts, mathematical proofs, step-by-step solved examples with code, practice questions with solutions, and technical interview drills.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for Data Structures & Algorithms (DSA) on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (10 CHAPTERS):

Ch 1: Asymptotic Analysis, Complexity Classes & Recurrence Relations

Ch 2: Arrays, Strings, Two Pointers & Sliding Window

Ch 3: Linked Lists & Fast-and-Slow Pointer Techniques

Ch 4: Stacks, Queues & Monotonic Data Structures

Ch 5: Binary Trees, Binary Search Trees & Balanced Trees

Ch 6: Heaps, Priority Queues & Greedy Algorithms

Ch 7: Graph Algorithms: Traversals, Shortest Paths & MST

Ch 8: Dynamic Programming Foundations & Classical Archetypes

Ch 9: Advanced Data Structures: Tries, DSU & Segment Trees

Ch 10: Solved Hard Placement DSA Problems & Interview Drills

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: Data Structures & Algorithms (DSA)

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Authoritative campus placement textbook covering Data Structures and Algorithms from foundational asymptotic complexity to advanced dynamic programming, graphs, tries, DSU, and solved FAANG coding problems.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: Asymptotic Analysis, Complexity Classes & Recurrence Relations](#chapter-1)
- [Chapter 2: Arrays, Strings, Two Pointers & Sliding Window](#chapter-2)
- [Chapter 3: Linked Lists & Fast-and-Slow Pointer Techniques](#chapter-3)
- [Chapter 4: Stacks, Queues & Monotonic Data Structures](#chapter-4)
- [Chapter 5: Binary Trees, Binary Search Trees & Balanced Trees](#chapter-5)
- [Chapter 6: Heaps, Priority Queues & Greedy Algorithms](#chapter-6)
- [Chapter 7: Graph Algorithms: Traversals, Shortest Paths & MST](#chapter-7)
- [Chapter 8: Dynamic Programming Foundations & Classical Archetypes](#chapter-8)
- [Chapter 9: Advanced Data Structures: Tries, DSU & Segment Trees](#chapter-9)
- [Chapter 10: Solved Hard Placement DSA Problems & Interview Drills](#chapter-10)

CHAPTER 1

Asymptotic Analysis, Complexity Classes & Recurrence Relations

Key Principle / Exam Takeaway: Master Big-O, Big-Omega, Big-Theta, recursion tree proofs, and the Master Theorem across all cases to evaluate any placement algorithm.

Core Curriculum Topics Covered in this Chapter:

- Formal definitions of Big-O, Big-Omega, Big-Theta
- Master Theorem for Divide-and-Conquer recurrences: T(n) = a*T(n/b) + f(n)
- Recursion tree method and substitution method proofs
- Space complexity: Auxiliary space vs call stack frame memory
- Complexity hierarchy: O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n) < O(n!)

· 1. Core Concepts & In-Depth Explanation

Asymptotic analysis is the mathematical foundation of algorithmic performance evaluation. Rather than measuring wall-clock time in seconds (which varies based on hardware architecture, background OS threads, and compiler optimizations), asymptotic analysis characterizes how execution time and memory space grow as input size n approaches infinity ($n \to \infty$).

In top corporate campus recruitment (Google, Amazon, Microsoft, Flipkart, Uber, TCS Digital), candidates are expected to rigorously justify the bounds of their algorithms:

- **Big-O (O):** Mathematical upper bound representing worst-case time or space.
- **Big-Omega (Ω):** Mathematical lower bound representing best-case performance.
- **Big-Theta (Θ):** Asymptotically tight bound holding when an algorithm's upper and lower bounds coincide.

When evaluating recursive algorithms, divide-and-conquer recurrences partition problems of size n into a subproblems each of size n/b, with $f(n)$ work spent splitting and merging. The Master Theorem provides an immediate closed-form solution by comparing $f(n)$ to the critical leaf-cost polynomial $n^{\log_b a}$.

· 2. Key Formulas, Rules & Complexity Classes

1. **Big-O Definition:** $$f(n) = O(g(n)) \iff \exists c > 0, n_0 \ge 0 \text{ s.t. } 0 \le f(n) \le c \cdot g(n) \quad \forall n \ge n_0$$ 2. **Big-Omega Definition:** $$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 \ge 0 \text{ s.t. } 0 \le c \cdot g(n) \le f(n) \quad \forall n \ge n_0$$ 3. **Big-Theta Definition:** $$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$ 4. **Master Theorem:** For $T(n) = a T(n/b) + f(n)$ where $a \ge 1, b > 1$:

- **Case 1 (Leaves Dominate):** If $f(n) = O(n^{\log_b a - \epsilon})$ for $\epsilon > 0 \implies T(n) = \Theta(n^{\log_b a})$
- **Case 2 (Even Balance):** If $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ for $k \ge 0 \implies T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$

- **Case 3 (Root Dominates):** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for $\epsilon > 0$ and $a f(n/b) \le c f(n)$ for $c < 1 \implies T(n) = \Theta(f(n))$

· 3. Step-by-Step Solved Examples

Example 1.1: Merge Sort Recurrence Derivation

- **Difficulty / Target Companies:** `Medium | Amazon, Microsoft, TCS Digital`
- **Problem Statement:** Solve the recurrence relation $T(n) = 2T(n/2) + cn$ with base case $T(1) = d$.

Step-by-Step Walkthrough & Logic:

- Step 1: Extract recurrence parameters: $a = 2$, $b = 2$, $f(n) = cn = \Theta(n^1)$.
- Step 2: Calculate the boundary exponent: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
- Step 3: Compare $f(n)$ with $n^{\log_b a}$. Both equal n, fitting Master Theorem Case 2 with $k = 0$.
- Step 4: Compute final complexity: $T(n) = \Theta(n^1 \cdot \log^{0+1} n) = \Theta(n \log n)$.

Complete Solved Code / Calculation:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

- **Line-by-Line Breakdown & Complexity Analysis:** At depth k of the recursion tree, there are 2^k subproblems each operating on arrays of size $n/2^k$. The work at depth k is $2^k \times c(n/2^k) = cn$. Since the tree has $\log_2 n$ levels, total work is $cn \log_2 n = \Theta(n \log n)$. Auxiliary space is $O(n)$ for temporary arrays.

Example 1.2: Binary Search Recurrence & Proof

- **Difficulty / Target Companies:** `Easy | Google, Infosys, Cognizant`
- **Problem Statement:** Prove that binary search on a sorted array of size n takes $O(\log n)$ time.

Step-by-Step Walkthrough & Logic:

- Step 1: In each iteration or recursive call, the search interval $[left, right]$ is halved: $T(n) = T(n/2) + O(1)$.

- Step 2: Master Theorem parameters: $a = 1, b = 2, f(n) = O(1) = \Theta(n^0)$.
- Step 3: $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 = \Theta(f(n))$.
- Step 4: Case 2 applies with $k = 0$: $T(n) = \Theta(1 \cdot \log^1 n) = \Theta(\log n)$.

Complete Solved Code / Calculation:

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2  # Safe calculation preventing 32-bit overflow
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

- **Line-by-Line Breakdown & Complexity Analysis:** Halving the search space repeatedly produces sequence $n, n/2, n/4, \dots, 1$. When $n/2^k = 1$, $2^k = n \implies k = \log_2 n$ iterations. Total time is strictly $O(\log n)$ with $O(1)$ auxiliary space.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: What is the tight bound Θ of $T(n) = 8T(n/2) + 1000n^2$?

Options:

- A) $\Theta(n^2)$
- B) $\Theta(n^3)$
- C) $\Theta(n^3 \log n)$
- D) $\Theta(n^2 \log n)$

Answer: **B) $\Theta(n^3)$** **Detailed Solution & Reasoning:** Here $a = 8, b = 2 \implies n^{\log_2 8} = n^3$. Since $f(n) = 1000n^2 = O(n^{3 - \epsilon})$ for $\epsilon = 1$, Master Theorem Case 1 applies. The leaf cost dominates: $T(n) = \Theta(n^3)$.

Practice Question 1.2

Question: Which of the following describes the space complexity of an in-order traversal of a balanced binary search tree with n nodes?

Options:

- A) $O(1)$ auxiliary space
- B) $O(\log n)$ call stack space
- C) $O(n)$ call stack space
- D) $O(n^2)$ space

Answer: **B) $O(\log n)$ call stack space** **Detailed Solution & Reasoning:** In a balanced binary tree, maximum recursion depth corresponds to tree height $h = \log_2 n$. Hence, the execution stack holds at most $O(\log n)$ activation records simultaneously.

Practice Question 1.3

Question: If an algorithm performs $1 + 2 + 3 + \dots + n$ operations, what is its asymptotic complexity?

Options:

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(2^n)$

Answer: **C) $O(n^2)$** **Detailed Solution & Reasoning:** The arithmetic sum $\sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} = \Theta(n^2)$.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Google)

- **Interview Question:** _Why is Quicksort preferred over Mergesort for internal sorting of arrays in C++ (std::sort), even though Quicksort has a worst-case of O(n^2)?_
- **Model High-Scoring Answer:** Quicksort has superior cache locality because it partitions elements in-place with contiguous memory reads. In contrast, Mergesort requires $O(n)$ auxiliary buffer allocation and continuous buffer copying, which incurs high cache miss penalties. Furthermore, randomized pivot selection or median-of-three heuristics makes Quicksort's worst-case probability infinitesimally small in practice ($O(n \log n)$ average with smaller constant factors).
- **Interviewer Follow-Up & Pro Tip:** ? Highlight hardware cache lines (L1/L2) and in-place partitioning mechanics.

Interview Question 1.2 (Amazon)

- **Interview Question:** _What is the difference between Amortized Complexity and Average Case Complexity?_
- **Model High-Scoring Answer:** Average-case complexity considers probabilistic assumptions over all possible input distributions (e.g., assuming any permutation is equally likely in Quicksort). In contrast, Amortized complexity makes zero probabilistic assumptions about inputs: it guarantees the average cost per operation across any arbitrary sequence of m operations in the worst case (e.g., dynamic array resizing where N appends cost $O(N)$ total, so amortized cost per append is strictly $O(1)$).
- **Interviewer Follow-Up & Pro Tip:** ? Cite the dynamic array vector doubling or aggregate accounting method to prove amortized $O(1)$.

Interview Question 1.3 (Microsoft)

- **Interview Question:** _How do you solve a recurrence relation where subproblem sizes decrease by a constant amount, such as T(n) = T(n-1) + n?_

- **Model High-Scoring Answer:** For linear recurrences where $T(n) = T(n-1) + f(n)$, the Master Theorem cannot be used. We apply the iterative unrolling method: $T(n) = T(n-1) + n = T(n-2) + (n-1) + n = \dots = T(1) + 2 + 3 + \dots + n = \Theta(n^2)$.

- **Interviewer Follow-Up & Pro Tip:** ? Mention that this exact recurrence governs the worst-case behavior of Quicksort when the array is already sorted and the extreme element is chosen as pivot.

CHAPTER 2

Arrays, Strings, Two Pointers & Sliding Window

Key Principle / Exam Takeaway: Master contiguous memory allocation, amortized doubling, opposite/same-directional two pointers, fixed/dynamic sliding window, and Kadane's algorithm.

Core Curriculum Topics Covered in this Chapter:

- Dynamic arrays memory allocation and amortized O(1) appending
- Opposite-directional two pointers (Two Sum II, Container With Most Water)
- Fast & slow pointers (In-place deduplication, partitioning)
- Fixed-size sliding window (Max sum subarray of size k)
- Variable-size sliding window with hash map (Longest substring without repeating characters)
- Kadane's algorithm for maximum contiguous subarray sum in O(n)

· 1. Core Concepts & In-Depth Explanation

Arrays provide contiguous physical memory storage, granting $O(1)$ random index access via base address arithmetic: $\text{Address}(arr[i]) = \text{Base} + i \times \text{SizeOfElement}$. Dynamic arrays automatically resize when capacity is reached, doubling the buffer size to achieve amortized $O(1)$ append time.

Two Pointers and Sliding Window are fundamental algorithmic archetypes that reduce brute-force $O(n^2)$ nested search spaces down to optimal $O(n)$ single-pass execution: 1. **Opposite-Directional Two Pointers:** Pointers initialize at array boundaries (`left = 0`, `right = n - 1`) and converge toward each other based on monotonic conditions. 2. **Same-Directional (Fast & Slow):** Pointers move in the same direction at varying rates to filter, deduplicate, or partition elements in-place with $O(1)$ auxiliary memory. 3. **Sliding Window:** Maintains a contiguous subarray sub-state. In fixed windows of size K, the window slides forward by adding the incoming element and subtracting the outgoing element in $O(1)$. In dynamic windows, the `right` pointer expands the window until a condition is violated, whereupon the `left` pointer contracts the window until validity is restored.

· 2. Key Formulas, Rules & Complexity Classes

1. **Amortized Analysis (Accounting Method):** Charging $\$3$ per insertion: $\$1$ pays for insertion into array; $\$2$ credit banked. When array doubles from N to $2N$, the N banked credits pay for copying all N elements into new memory. Amortized cost $= O(1)$. 2. **Fixed Sliding Window Sum Invariant:** $$\text{WindowSum}_{i} = \text{WindowSum}_{i-1} + arr[i] - arr[i - K]$$ 3. **Kadane's Dynamic Recurrence:** $$\text{CurrentMax}[i] = \max(arr[i], \text{CurrentMax}[i-1] + arr[i])$$ $$\text{GlobalMax} = \max_{0 \le i < n}(\text{CurrentMax}[i])$$

· 3. Step-by-Step Solved Examples

Example 2.1: Container With Most Water (Optimal Two Pointers)

- **Difficulty / Target Companies:** `Medium | Google, Amazon, Flipkart`

- **Problem Statement:** Given an integer array height of length n, find two lines that together with the x-axis form a container that contains the most water. Return the maximum area.

Step-by-Step Walkthrough & Logic:

- Step 1: The area between two boundaries at index i and j is $\text{Area} = (j - i) \times \min(height[i], height[j])$.
- Step 2: Start with maximum width: `left = 0`, `right = n - 1`.
- Step 3: At each step, the area is bounded by the shorter line. Moving the taller pointer cannot increase the area because width decreases while height remains limited by the shorter line.
- Step 4: Therefore, greedily advance the pointer pointing to the shorter vertical bar.

Complete Solved Code / Calculation:

```
def max_area(height):
    left, right = 0, len(height) - 1
    max_water = 0
    while left < right:
        width = right - left
        h = min(height[left], height[right])
        max_water = max(max_water, width * h)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return max_water
```

- **Line-by-Line Breakdown & Complexity Analysis:** Since every step advances one pointer inward, the loop executes at most n times. Time complexity is strictly $O(n)$ with $O(1)$ auxiliary space, compared to the brute force $O(n^2)$ approach.

Example 2.2: Longest Substring Without Repeating Characters (Dynamic Window)

- **Difficulty / Target Companies:** `Medium | Microsoft, Uber, TCS Digital`
- **Problem Statement:** Given a string `s`, find the length of the longest substring without repeating characters.

Step-by-Step Walkthrough & Logic:

- Step 1: Maintain a hash map `char_index` storing the most recent 0-based index of each character encountered.
- Step 2: Initialize `left = 0` and `max_len = 0`.
- Step 3: Iterate `right` from 0 to len(s) - 1. If `s[right]` is already in `char_index` and its recorded position is $\ge left$, jump `left = char_index[s[right]] + 1`.
- Step 4: Update `char_index[s[right]] = right` and calculate `max_len = max(max_len, right - left + 1)`.

Complete Solved Code / Calculation:

```
def length_of_longest_substring(s: str) -> int:
    char_index = {}
    left = 0
    max_len = 0
    for right, ch in enumerate(s):
        if ch in char_index and char_index[ch] >= left:
```

```
            left = char_index[ch] + 1
        char_index[ch] = right
        max_len = max(max_len, right - left + 1)
    return max_len
```

- **Line-by-Line Breakdown & Complexity Analysis:** The right pointer visits each character once. The left pointer only jumps forward, never backward. Time complexity is $O(n)$. Auxiliary space is $O(\min(n, m))$ where m is the character alphabet size (at most 128 for ASCII).

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: What is the time and space complexity of Kadane's algorithm for finding the maximum subarray sum?

Options:

- A) $O(n)$ time, $O(n)$ space
- B) $O(n)$ time, $O(1)$ space
- C) $O(n \log n)$ time, $O(1)$ space
- D) $O(n^2)$ time, $O(1)$ space

Answer: **B) $O(n)$ time, $O(1)$ space** **Detailed Solution & Reasoning:** Kadane's algorithm scans the array in a single pass maintaining two scalar variables: `current_sum` and `max_sum`. Thus it requires strictly $O(n)$ time and $O(1)$ auxiliary space.

Practice Question 2.2

Question: In the Two Sum II problem on a sorted array, if `arr[left] + arr[right] > target`, what is the optimal next move?

Options:

- A) Increment `left`
- B) Decrement `right`
- C) Reset `left = 0`
- D) Break the loop

Answer: **B) Decrement `right`** **Detailed Solution & Reasoning:** Because the array is sorted in ascending order, decreasing `right` reduces the sum, moving it closer to the target.

Practice Question 2.3

Question: When does the amortized cost of an operation differ from its worst-case cost?

Options:

- A) Only when recursion is used
- B) When occasional expensive operations are offset by numerous cheap operations
- C) When space complexity exceeds $O(n)$

- D) Amortized cost is always identical to worst-case cost

Answer: **B) When occasional expensive operations are offset by numerous cheap operations** **Detailed Solution & Reasoning:** Dynamic array resizing takes $O(n)$ in the worst case, but occurs so infrequently that across a sequence of n operations, the average cost per operation is strictly $O(1)$.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Amazon)

- **Interview Question:** _How would you handle negative numbers when applying the Sliding Window technique to find a subarray with sum equal to K?_
- **Model High-Scoring Answer:** The standard two-pointer sliding window relies on monotonicity (expanding increases sum, contracting decreases sum). When negative numbers exist, monotonicity is violated. Instead, we use Prefix Sums with a Hash Map. We maintain running `prefix_sum`. If `prefix_sum - K` exists in our hash map, the subarray between that recorded index and the current index sums exactly to K. This runs in $O(n)$ time and $O(n)$ space.
- **Interviewer Follow-Up & Pro Tip:** ? Identifying when sliding window breaks and seamlessly transitioning to prefix sums + hash maps is an instant hire signal.

Interview Question 2.2 (Flipkart)

- **Interview Question:** _Explain how to remove duplicates from a sorted array in-place with $O(1)$ extra memory._
- **Model High-Scoring Answer:** Use two pointers: a slow pointer `write_idx = 1` and a fast pointer `i` iterating from 1 to $n-1$. Compare `arr[i]` with `arr[write_idx - 1]`. If they are different, copy `arr[write_idx] = arr[i]` and increment `write_idx`. At the end of the loop, `write_idx` holds the length of the deduplicated array.
- **Interviewer Follow-Up & Pro Tip:** ? State clearly that the first element is always unique, so indices start at 1.

Interview Question 2.3 (Microsoft)

- **Interview Question:** _Given an array of integers representing stock prices, how does Kadane's algorithm relate to finding the maximum single-trade profit?_
- **Model High-Scoring Answer:** The maximum stock profit problem ($\max(prices[j] - prices[i])$ for $j > i$) can be transformed into the maximum subarray sum problem by taking differences between consecutive days: $\Delta[k] = prices[k] - prices[k-1]$. The sum of differences $\sum_{k=i+1}^j \Delta[k] = prices[j] - prices[i]$ telescopes. Finding the maximum contiguous subarray sum of Δ using Kadane's algorithm yields the maximum profit in $O(n)$ time and $O(1)$ space.
- **Interviewer Follow-Up & Pro Tip:** ? Mentioning the telescoping sum equivalence showcases deep mathematical maturity.

CHAPTER 3

Linked Lists & Fast-and-Slow Pointer Techniques

Key Principle / Exam Takeaway: Master node reference manipulation, in-place reversal, Floyd's Cycle Detection proof, fast/slow mid-finding, and LRU Cache doubly linked list architecture.

Core Curriculum Topics Covered in this Chapter:

- Singly vs doubly linked list memory models and pointer manipulation
- In-place iterative linked list reversal in O(n) time and O(1) space
- Floyd's Cycle Detection Algorithm (Tortoise and Hare) and mathematical proof
- Finding the middle node and palindromic linked list verification
- LRU Cache design using Hash Map + Doubly Linked List with O(1) get/put

· 1. Core Concepts & In-Depth Explanation

Linked lists represent non-contiguous linear collections where each node stores a payload and one or more references (`next`, `prev`) to adjacent nodes. Unlike arrays, linked lists allow $O(1)$ insertions and deletions at known pointer positions without requiring element shifting, at the expense of losing $O(1)$ random indexing.

Floyd's Cycle Finding Algorithm (Tortoise and Hare) is an essential placement pattern. Two pointers traverse the list: `slow` advances 1 step per cycle while `fast` advances 2 steps. If a cycle exists, `fast` must enter the cycle and catch up to `slow` from behind because the distance between them shrinks by exactly 1 node per iteration. Once they collide, resetting `slow` to the head and advancing both pointers at 1 step per iteration leads directly to the cycle origin node.

· 2. Key Formulas, Rules & Complexity Classes

1. **Floyd's Cycle Distance Proof:** Let distance from head to cycle entrance $= L_1$. Let distance from cycle entrance to collision point $= L_2$. Let cycle perimeter $= C$. $$\text{Distance}(\text{slow}) = L_1 + L_2$$ $$\text{Distance}(\text{fast}) = L_1 + L_2 + k \cdot C$$ Since $\text{speed}(\text{fast}) = 2 \times \text{speed}(\text{slow})$: $$2(L_1 + L_2) = L_1 + L_2 + k \cdot C \implies L_1 + L_2 = k \cdot C \implies L_1 = k \cdot C - L_2$$ Hence, advancing a pointer from the head and one from the collision point at equal speeds causes them to meet at the cycle entrance!

· 3. Step-by-Step Solved Examples

Example 3.1: Reverse a Singly Linked List In-Place

- **Difficulty / Target Companies:** `Easy | Google, Amazon, TCS`
- **Problem Statement:** Given the head of a singly linked list, reverse the list in-place and return the reversed list head.

Step-by-Step Walkthrough & Logic:

- Step 1: Maintain three pointers: `prev = None`, `curr = head`, and `next_temp = None`.
- Step 2: While `curr` is not null: save `next_temp = curr.next`.
- Step 3: Reverse the current node pointer: `curr.next = prev`.

- Step 4: Shift pointers forward: `prev = curr`, `curr = next_temp`.
- Step 5: Return `prev` as the new head.

Complete Solved Code / Calculation:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list(head: ListNode) -> ListNode:
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

- **Line-by-Line Breakdown & Complexity Analysis:** Each node pointer is modified exactly once. Time complexity is $O(n)$ and auxiliary space is $O(1)$ because no new nodes are allocated.

Example 3.2: Detect Cycle & Find Cycle Origin (Floyd's Algorithm)

- **Difficulty / Target Companies:** `Medium | Microsoft, Uber, Adobe`
- **Problem Statement:** Given head of a linked list, return the node where the cycle begins. If no cycle exists, return null.

Step-by-Step Walkthrough & Logic:

- Step 1: Initialize `slow = fast = head`.
- Step 2: Advance `slow` by 1 and `fast` by 2 until `fast` is null or `slow == fast`.
- Step 3: If `fast` reaches null, no cycle exists; return null.
- Step 4: If collision occurs, reset `slow = head`.
- Step 5: Advance both `slow` and `fast` by 1 step simultaneously. Their meeting node is the cycle start.

Complete Solved Code / Calculation:

```
def detect_cycle(head: ListNode) -> ListNode:
    slow = fast = head
    has_cycle = False
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            has_cycle = True
            break
    if not has_cycle:
        return None
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next
    return slow
```

- **Line-by-Line Breakdown & Complexity Analysis:** Finding the collision takes $O(n)$ time. Locating the entrance takes $L_1 \le n$ steps. Total time is $O(n)$ with $O(1)$ auxiliary space.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: What is the time complexity to find the middle element of a linked list using the fast and slow pointer technique?

Options:

- A) $O(n)$ time, $O(n)$ space
- B) $O(n)$ time, $O(1)$ space
- C) $O(n^2)$ time, $O(1)$ space
- D) $O(\log n)$ time, $O(1)$ space

Answer: **B) $O(n)$ time, $O(1)$ space** **Detailed Solution & Reasoning:** Fast pointer traverses n nodes while slow traverses $n/2$ nodes in a single pass. Auxiliary memory consists of two pointers, taking strictly $O(1)$ space.

Practice Question 3.2

Question: Why is a Doubly Linked List paired with a Hash Map in an LRU Cache implementation?

Options:

- A) To achieve $O(1)$ random indexing
- B) To allow $O(1)$ node removal and insertion at both head and tail given node reference
- C) To save memory over arrays
- D) To automatically sort keys

Answer: **B) To allow $O(1)$ node removal and insertion at both head and tail given node reference** **Detailed Solution & Reasoning:** In a singly linked list, deleting a node requires finding its predecessor in $O(n)$. A doubly linked list node stores `prev`, enabling $O(1)$ disconnection and relocation to the most-recently-used position.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Amazon)

- **Interview Question:** _Design an LRU Cache with get(key) and put(key, value) operations executing in O(1) time._
- **Model High-Scoring Answer:** Use a Hash Map mapping `key` to `Node(key, value)` and a Doubly Linked List with dummy `head` and `tail` sentinels. For `get(key)`: look up node in hash map; if present, unlink node and move it to `head.next` (most recently used), returning value. For `put(key, value)`: if key exists, update value and move to head. If key does not exist and cache is at capacity, remove the least recently used node at `tail.prev`, erase it from hash map, and insert new node at head.

- **Interviewer Follow-Up & Pro Tip:** ? Always mention dummy head and tail sentinel nodes because they eliminate null checks on edge insertions/deletions.

CHAPTER 4

Stacks, Queues & Monotonic Data Structures

Key Principle / Exam Takeaway: Master LIFO/FIFO mechanics, expression evaluation, monotonic stack pattern for Next Greater Element, and monotonic deque for sliding window maximum.

Core Curriculum Topics Covered in this Chapter:

- Stack & Queue abstract data types and array/linked-list backing
- Infix, prefix, and postfix expression evaluation and parenthesis balancing
- Monotonic Stack pattern: Next Greater Element and Next Smaller Element in O(n)
- Largest Rectangle in Histogram solved in O(n) via monotonic stack
- Sliding Window Maximum in O(n) via monotonic doubly-ended queue (deque)

· 1. Core Concepts & In-Depth Explanation

Stacks (Last-In, First-Out: LIFO) and Queues (First-In, First-Out: FIFO) are core linear data structures managing constrained access orders. Stacks power runtime execution calls, depth-first searches, and backtracking, while Queues manage task scheduling, buffering, and breadth-first search.

The **Monotonic Stack / Deque** is an advanced pattern crucial for top-tier placement assessments. A monotonic stack maintains elements in strictly increasing or decreasing order. As each element is processed, existing elements that violate the monotonic invariant are popped off. Although elements are pushed and popped inside loops, every element enters and exits the stack at most once, delivering strictly $O(n)$ amortized running time across the entire array.

· 2. Key Formulas, Rules & Complexity Classes

1. **Monotonic Stack Invariant (Decreasing Stack for Next Greater Element):** When processing element x: $$\text{while } (\text{stack not empty} \text{ and } \text{stack.top}() < x): \quad \text{pop}() \implies \text{NextGreater}(\text{popped}) = x$$ 2. **Amortized Time Proof:** Each of n elements is pushed once and popped at most once. Total push operations $= n$, total pop operations $\le n$. Total work $\le 2n = O(n)$.

· 3. Step-by-Step Solved Examples

Example 4.1: Next Greater Element I

- **Difficulty / Target Companies:** `Medium | Amazon, Flipkart, Goldman Sachs`
- **Problem Statement:** Given an array of integers, find the next greater element for each item. If no greater element exists to the right, return -1.

Step-by-Step Walkthrough & Logic:

- Step 1: Initialize an array `result` of size n filled with -1, and an empty stack `st`.
- Step 2: Iterate through the array with index `i` from 0 to $n-1$.
- Step 3: While `st` is not empty and `arr[i] > arr[st[-1]]`: pop index `idx = st.pop()` and record `result[idx] = arr[i]`.

- Step 4: Push current index `i` onto `st`.
- Step 5: Remaining elements in stack have no greater element to their right (remain -1).

Complete Solved Code / Calculation:

```
def next_greater_elements(nums):
    n = len(nums)
    res = [-1] * n
    stack = []  # Stores indices
    for i in range(n):
        while stack and nums[i] > nums[stack[-1]]:
            idx = stack.pop()
            res[idx] = nums[i]
        stack.append(i)
    return res
```

- **Line-by-Line Breakdown & Complexity Analysis:** Each index is pushed once and popped once. Overall time complexity is strictly $O(n)$ with $O(n)$ auxiliary space.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: What is the time complexity of finding the Largest Rectangle in Histogram using a monotonic stack?

Options:

- A) $O(n^2)$
- B) $O(n \log n)$
- C) $O(n)$
- D) $O(n^3)$

Answer: **C) $O(n)$** **Detailed Solution & Reasoning:** By maintaining an increasing monotonic stack of bar indices, each bar is pushed and popped at most once to determine left and right boundaries, yielding $O(n)$ time.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Google)

- **Interview Question:** _How does a Monotonic Deque solve Sliding Window Maximum in O(n) compared to a Priority Queue in O(n log k)?_
- **Model High-Scoring Answer:** A Priority Queue requires $O(\log k)$ to push and pop window elements, resulting in $O(n \log k)$ total time. A Monotonic Deque stores indices of elements in descending order. When sliding the window: 1) Pop indices from the front that have fallen out of the window (`dq[0] < i - k + 1`). 2) Pop indices from the back whose values are $\le arr[i]$ since they can never be the maximum again. 3) Push `i` to back. The maximum of the current window is always at `dq[0]`. Since every index enters and leaves the deque at most once, total time is strictly $O(n)$.
- **Interviewer Follow-Up & Pro Tip:** ? Mention that deque stores indices rather than values so expired window items can be identified in $O(1)$.

CHAPTER 5

Binary Trees, Binary Search Trees & Balanced Trees

Key Principle / Exam Takeaway: Master tree traversals (in-order, pre-order, post-order, level-order), BST invariant, Lowest Common Ancestor, and self-balancing AVL/Red-Black rotation proofs.

Core Curriculum Topics Covered in this Chapter:

- Tree properties: Height, depth, diameter, full vs complete vs perfect trees
- Traversals: Recursive vs iterative (Morris Traversal for O(1) space)
- Binary Search Tree operations: Insertion, deletion with in-order successor, search
- Lowest Common Ancestor (LCA) in Binary Trees and BSTs
- Balanced Trees: AVL rotation mechanics (LL, RR, LR, RL) and Red-Black properties

· 1. Core Concepts & In-Depth Explanation

A Binary Tree is a hierarchical non-linear structure where every node has at most two children (`left`, `right`). Binary Search Trees (BST) enforce the fundamental ordering property: for any node X, all keys in X's left subtree are strictly less than $X.val$, and all keys in X's right subtree are strictly greater than $X.val$. This enables $O(h)$ search, insertion, and deletion, where h is the tree height.

If elements are inserted in sorted order, an un-balanced BST degenerates into a linear linked list of height $h = n$, degrading search time to $O(n)$. Self-balancing trees (AVL Trees and Red-Black Trees) dynamically maintain logarithmic height $h = O(\log n)$ via tree rotations (Left Rotation and Right Rotation). In-order traversal of any valid BST yields all keys in strictly ascending sorted order.

· 2. Key Formulas, Rules & Complexity Classes

1. **Tree Properties:**

- In a binary tree of height h (where root height $= 0$): Maximum nodes $= 2^{h+1} - 1$.
- For n nodes, minimum height $= \lfloor \log_2 n \rfloor$.

2. **AVL Balance Factor:** $$\text{BalanceFactor}(N) = \text{Height}(\text{left}) - \text{Height}(\text{right}) \in \{-1, 0, +1\}$$ 3. **Lowest Common Ancestor in BST:** If $\min(p.val, q.val) \le root.val \le \max(p.val, q.val)$, then $root$ is the LCA!

· 3. Step-by-Step Solved Examples

Example 5.1: Lowest Common Ancestor in a Binary Search Tree

- **Difficulty / Target Companies:** `Easy | Amazon, Microsoft, TCS`
- **Problem Statement:** Given a BST and two nodes p and q, find their Lowest Common Ancestor (LCA).

Step-by-Step Walkthrough & Logic:

- Step 1: Start at the root node.

- Step 2: If both p and q have values less than root, LCA must reside in the left subtree: recurse left.
- Step 3: If both p and q have values greater than root, LCA must reside in the right subtree: recurse right.
- Step 4: If p and q diverge (one is on left, one is on right, or one equals root), the current node is the split point and hence the LCA.

Complete Solved Code / Calculation:

```
def lowest_common_ancestor_bst(root, p, q):
    curr = root
    while curr:
        if p.val < curr.val and q.val < curr.val:
            curr = curr.left
        elif p.val > curr.val and q.val > curr.val:
            curr = curr.right
        else:
            return curr
    return None
```

- **Line-by-Line Breakdown & Complexity Analysis:** The algorithm visits at most one node per level. In a balanced BST of height $h = \log n$, time complexity is $O(\log n)$ with $O(1)$ auxiliary space.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: What is the maximum number of nodes at level k of a binary tree (root at level 0)?

Options:

- A) 2^k
- B) 2^{k-1}
- C) $2k$
- D) k^2

Answer: **A) 2^k** **Detailed Solution & Reasoning:** Level 0 has $2^0 = 1$ node. Level 1 has $2^1 = 2$. Level k has at most 2^k nodes.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Amazon)

- **Interview Question:** _How do you validate whether a given binary tree is a valid Binary Search Tree?_
- **Model High-Scoring Answer:** Do not simply check if `root.left.val < root.val < root.right.val` because that only checks immediate children, not entire subtrees. Instead, carry a valid range `(min_val, max_val)` through a recursive DFS: for root, range is $(-\infty, +\infty)$. When recurring left, range becomes `(min_val, root.val)`. When recurring right, range becomes `(root.val, max_val)`. If any node violates `min_val < node.val < max_val`, return False. This checks the tree in $O(n)$ time and $O(h)$ space.
- **Interviewer Follow-Up & Pro Tip:** ? Mention that an in-order traversal of a valid BST must produce a strictly increasing sequence.

CHAPTER 6

Heaps, Priority Queues & Greedy Algorithms

Key Principle / Exam Takeaway: Master binary heap array storage, heapify proof of O(n) construction, Top-K elements, Median from Data Stream (two heaps), and greedy exchange arguments.

Core Curriculum Topics Covered in this Chapter:

- Complete binary tree representation in contiguous arrays
- Min-heap and Max-heap invariants: Sift-up and sift-down mechanics
- Mathematical proof of O(n) bottom-up heap construction vs O(n log n) naive insertion
- Find Median from Data Stream using Two Heaps (Max-Heap + Min-Heap)
- Greedy choice property, optimal substructure, and activity selection

· 1. Core Concepts & In-Depth Explanation

A Binary Heap is a complete binary tree satisfying the Heap Property: in a Min-Heap, every parent node is $\le$ its children (root holds the minimum element); in a Max-Heap, every parent node is $\ge$ its children. Because complete binary trees have no missing nodes except on the bottom row from left to right, heaps are stored without pointers in a compact contiguous array:

- Parent of node at index i: $\lfloor (i - 1) / 2 \rfloor$
- Left child of index i: $2i + 1$
- Right child of index i: $2i + 2$

Building a heap from an unordered array of n elements bottom-up via `build_heap()` runs in strictly $O(n)$ time?not $O(n \log n)$?because the majority of nodes reside near the leaves where sift-down height is tiny.

· 2. Key Formulas, Rules & Complexity Classes

1. **Mathematical Proof of O(n) Heap Construction:** Let $h = \log_2 n$. At height k, there are at most $\lceil n / 2^{k+1} \rceil$ nodes, each requiring at most k operations: $$T(n) = \sum_{k=0}^{\lfloor \log n \rfloor} \frac{n}{2^{k+1}} \times k = \frac{n}{2} \sum_{k=0}^\infty \frac{k}{2^k}$$ Since the arithmetico-geometric series $\sum_{k=0}^\infty \frac{k}{2^k} = 2$: $$T(n) = \frac{n}{2} \times 2 = O(n)$$

· 3. Step-by-Step Solved Examples

Example 6.1: Find Kth Largest Element in an Array

- **Difficulty / Target Companies:** `Medium | Amazon, Facebook, Google`
- **Problem Statement:** Find the k-th largest element in an unsorted array of size n.

Step-by-Step Walkthrough & Logic:

- Step 1: Maintain a Min-Heap of capacity k.

- Step 2: Iterate through each element in the array.
- Step 3: Push current element to min-heap. If heap size exceeds k, pop the root (the smallest of the $k+1$ items).
- Step 4: After processing all n items, the min-heap contains the k largest items, with the k-th largest at the root.

Complete Solved Code / Calculation:

```
import heapq

def find_kth_largest(nums, k):
    min_heap = []
    for x in nums:
        heapq.heappush(min_heap, x)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return min_heap[0]
```

- **Line-by-Line Breakdown & Complexity Analysis:** Heap size never exceeds $k+1$. Pushing and popping takes $O(\log k)$. For n items, total time is $O(n \log k)$ with $O(k)$ auxiliary space, far outperforming $O(n \log n)$ full sorting when $k \ll n$.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: What is the time complexity to insert an element into a binary heap containing n elements?

Options:

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Answer: **B) $O(\log n)$** **Detailed Solution & Reasoning:** Inserting an element appends it to the end and sifts it up to the root, which requires traversing at most the tree height $h = \log_2 n$ levels.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Google)

- **Interview Question:** _How do you find the running median from a continuous stream of numbers in real time?_
- **Model High-Scoring Answer:** Use two heaps: a Max-Heap `small` storing the smaller half of numbers, and a Min-Heap `large` storing the larger half. Maintain the balance invariant: `len(small) == len(large)` or `len(small) == len(large) + 1`. For incoming number x: push to `small`, then pop max from `small` and push to `large`. If `len(large) > len(small)`, pop min from `large` and push to `small`. Finding median takes $O(1)$: if lengths are equal, median is `(small[0] + large[0]) / 2.0`; otherwise median is `small[0]`. Insertion is $O(\log n)$.
- **Interviewer Follow-Up & Pro Tip:** ? Clearly articulate the two heap balancing invariants.

CHAPTER 7

Graph Algorithms: Traversals, Shortest Paths & MST

Key Principle / Exam Takeaway: Master Adjacency Matrix vs List, BFS/DFS, topological sort (Kahn's), Dijkstra's algorithm, Bellman-Ford negative cycle detection, and Kruskal's MST.

Core Curriculum Topics Covered in this Chapter:

- Graph representations: Adjacency Matrix (O(V^2)) vs Adjacency List (O(V + E))
- Breadth-First Search (BFS) and Depth-First Search (DFS) dynamics
- Cycle detection in directed graphs & Topological Sorting (Kahn's Algorithm)
- Single-source shortest paths: Dijkstra's Algorithm (O((V + E) log V)) vs Bellman-Ford
- Minimum Spanning Trees: Kruskal's Algorithm using Disjoint Set Union (DSU) vs Prim's

· 1. Core Concepts & In-Depth Explanation

A Graph $G = (V, E)$ consists of vertices V connected by edges E. In placement evaluations, choosing between an Adjacency Matrix and an Adjacency List is critical: Adjacency Matrices take $O(V^2)$ space and are ideal only for dense graphs, whereas Adjacency Lists take $O(V + E)$ space and are optimal for sparse networks.

Graph traversal forms the core of network analysis:

- **BFS (Queue):** Discovers vertices level-by-level, identifying shortest paths in unweighted graphs in $O(V + E)$.
- **DFS (Stack/Recursion):** Explores deeply along branches, fundamental for cycle detection, connected components, and topological sorting.
- **Topological Sort:** Linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge $u \to v$, u appears before v. Kahn's algorithm computes this using in-degrees: vertices with in-degree 0 are queued and processed iteratively.

· 2. Key Formulas, Rules & Complexity Classes

1. **Dijkstra's Priority Queue Time Complexity:** With binary min-heap: each vertex is extracted once ($V \log V$) and each edge relaxes at most once ($E \log V$): $$T(V, E) = O((V + E) \log V)$$ 2. **Bellman-Ford Invariant:** Relax all E edges $V - 1$ times. If any edge can still be relaxed on the V-th pass, a negative weight cycle exists!

· 3. Step-by-Step Solved Examples

Example 7.1: Course Schedule (Topological Sort / Cycle Detection)

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Microsoft`
- **Problem Statement:** Given n courses labelled 0 to $n-1$ and prerequisite pairs $[a, b]$ meaning you must take course b before a, determine if it is possible to finish all courses.

Step-by-Step Walkthrough & Logic:

- Step 1: Build adjacency list and compute in-degree for all n nodes.
- Step 2: Initialize a queue containing all nodes with `in_degree == 0`.
- Step 3: While queue is not empty: pop node `u`, increment `processed_count` by 1. For each neighbor `v` of `u`, decrement `in_degree[v]`. If `in_degree[v] == 0`, push `v` to queue.
- Step 4: If `processed_count == n`, all courses can be finished (no cycle). Otherwise, a cycle exists.

Complete Solved Code / Calculation:

```
from collections import deque

def can_finish(num_courses, prerequisites):
    adj = {i: [] for i in range(num_courses)}
    in_degree = [0] * num_courses
    for dest, src in prerequisites:
        adj[src].append(dest)
        in_degree[dest] += 1

    queue = deque([i for i in range(num_courses) if in_degree[i] == 0])
    count = 0
    while queue:
        node = queue.popleft()
        count += 1
        for nbr in adj[node]:
            in_degree[nbr] -= 1
            if in_degree[nbr] == 0:
                queue.append(nbr)
    return count == num_courses
```

- **Line-by-Line Breakdown & Complexity Analysis:** Every node enters the queue once and every edge is decremented once. Time complexity is $O(V + E)$ with $O(V + E)$ space.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: Why can Dijkstra's algorithm fail on graphs with negative edge weights?

Options:

- A) It causes an infinite loop
- B) The greedy choice assumption fails because a finalized node's distance could be decreased later via a negative edge
- C) It cannot use an adjacency list
- D) It only works on trees

Answer: **B) The greedy choice assumption fails because a finalized node's distance could be decreased later via a negative edge** **Detailed Solution & Reasoning:** Dijkstra assumes that extracting the minimum distance vertex locks in its optimal shortest path. Negative edges violate this monotonic property.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Microsoft)

- **Interview Question:** _What is the difference between Prim's algorithm and Kruskal's algorithm for finding Minimum Spanning Trees?_
- **Model High-Scoring Answer:** Both find a Minimum Spanning Tree of weight sum minimum in a connected weighted undirected graph. Prim's algorithm is vertex-centric: it grows a single connected tree outward from an arbitrary start node using a priority queue, running in $O(E \log V)$ time (superior for dense graphs). Kruskal's algorithm is edge-centric: it sorts all E edges globally ($O(E \log E)$) and greedily adds edges that do not form cycles using Disjoint Set Union (DSU with union-by-rank and path compression, taking $O(E \alpha(V))$). Kruskal's is simpler and faster on sparse graphs.
- **Interviewer Follow-Up & Pro Tip:** ? Mention Disjoint Set Union (DSU) and the inverse Ackermann function $\alpha(V)$.

CHAPTER 8

Dynamic Programming Foundations & Classical Archetypes

Key Principle / Exam Takeaway: Master optimal substructure, overlapping subproblems, memoization vs tabulation, 0/1 Knapsack, Longest Common Subsequence, and 1D space optimization.

Core Curriculum Topics Covered in this Chapter:

- Prerequisites for DP: Optimal Substructure & Overlapping Subproblems
- Top-Down Memoization vs Bottom-Up Tabulation state transitions
- 0/1 Knapsack Problem and 1D array space optimization
- Longest Common Subsequence (LCS) and Edit Distance (Levenshtein)
- Longest Increasing Subsequence (LIS) in O(n log n) via Patience Sorting

· 1. Core Concepts & In-Depth Explanation

Dynamic Programming (DP) solves complex optimization problems by decomposing them into overlapping subproblems and storing subproblem results to prevent redundant re-computation. A problem qualifies for DP if it satisfies two conditions: 1. **Optimal Substructure:** The optimal solution to the overall problem incorporates optimal solutions to its constituent subproblems. 2. **Overlapping Subproblems:** The recursive solution visits the same identical subproblem states repeatedly.

There are two primary paradigms:

- **Top-Down with Memoization:** Starts from the original problem and recurses downward, caching computed results in a hash table or array.
- **Bottom-Up Tabulation:** Iteratively builds solutions starting from the smallest base cases, avoiding recursive call stack overhead and enabling space optimization (e.g., rolling array or 1D buffer reduction).

· 2. Key Formulas, Rules & Complexity Classes

1. **0/1 Knapsack State Recurrence:** Let $dp[i][w]$ be the max value using a subset of first i items with weight capacity w: $$dp[i][w] = \begin{cases} dp[i-1][w] & \text{if } wt[i-1] > w \\ \max(dp[i-1][w], \ val[i-1] + dp[i-1][w - wt[i-1]]) & \text{if } wt[i-1] \le w \end{cases}$$ 2. **1D Space Optimization:** Iterate w backwards from W down to $wt[i-1]$ to ensure each item is used at most once: $$dp[w] = \max(dp[w], \ val[i-1] + dp[w - wt[i-1]])$$

· 3. Step-by-Step Solved Examples

Example 8.1: 0/1 Knapsack Problem with 1D Space Optimization

- **Difficulty / Target Companies:** `Medium | Amazon, Flipkart, Goldman Sachs`
- **Problem Statement:** Given weights wt, values val, and max capacity W, find the maximum value subset fitting in capacity W.

Step-by-Step Walkthrough & Logic:

- Step 1: Initialize 1D array `dp = [0] * (W + 1)`.
- Step 2: Iterate through each item index i from 0 to $n-1$.

- Step 3: Iterate capacity w backwards from W down to $wt[i]$. (Iterating backwards ensures values from the current item are not reused).
- Step 4: Update `dp[w] = max(dp[w], val[i] + dp[w - wt[i]])`.
- Step 5: Return `dp[W]`.

Complete Solved Code / Calculation:

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [0] * (W + 1)
    for i in range(n):
        w_curr = weights[i]
        v_curr = values[i]
        for w in range(W, w_curr - 1, -1):
            dp[w] = max(dp[w], v_curr + dp[w - w_curr])
    return dp[W]
```

- **Line-by-Line Breakdown & Complexity Analysis:** Time complexity is $O(n \times W)$ (pseudo-polynomial). Space complexity is reduced from $O(n \times W)$ to strictly $O(W)$.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: Why is the 0/1 Knapsack problem classified as NP-Complete rather than strictly polynomial time?

Options:

- A) Because it cannot be solved with recursion
- B) Because running time $O(n W)$ is polynomial in the numeric value of W, but exponential in the input size (number of bits $\log W$)
- C) Because it requires sorting
- D) Because negative values are disallowed

Answer: **B) Because running time $O(n W)$ is polynomial in the numeric value of W, but exponential in the input size (number of bits $\log W$)** **Detailed Solution & Reasoning:** Algorithms whose running times depend on numeric input magnitude rather than length in bits are called pseudo-polynomial.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Google)

- **Interview Question:** _How do you optimize Longest Increasing Subsequence (LIS) from O(n^2) to O(n log n)?_
- **Model High-Scoring Answer:** Use Patience Sorting with Binary Search. Maintain an array `tails` where `tails[i]` stores the smallest tail of all increasing subsequences of length $i+1$ found so far. For each number x in the input: binary search for the first element in `tails` $\ge x$. If found, replace it with x (greedily lowering the tail value for that length). If x is greater than all elements in `tails`, append x. The length of `tails` at the end is the length of the LIS. For n elements with binary search taking $O(\log n)$, total time is strictly $O(n \log n)$ with $O(n)$ space.
- **Interviewer Follow-Up & Pro Tip:** ? Emphasize that `tails` is always sorted, which validates the binary search step.

CHAPTER 9

Advanced Data Structures: Tries, DSU & Segment Trees

Key Principle / Exam Takeaway: Master Prefix Trees (Trie), Disjoint Set Union with Path Compression and Union by Rank, and Segment Trees for O(log n) range query and point updates.

Core Curriculum Topics Covered in this Chapter:

- Trie (Prefix Tree) implementation: insert, search, startsWith in O(L)
- Disjoint Set Union (DSU): Path compression and union-by-rank proof of O(alpha(n))
- Segment Tree architecture: Build in O(n), Range Query in O(log n), Update in O(log n)
- Fenwick Tree (Binary Indexed Tree) for range prefix sums with O(n) space

· 1. Core Concepts & In-Depth Explanation

Advanced data structures handle specialized high-throughput requirements in enterprise architectures: 1. **Trie (Prefix Tree):** An m-ary tree where nodes store characters along word paths. It enables $O(L)$ word insertion, search, and prefix matching (where L is word length), independent of dictionary size. Powers autocomplete engines and search suggestion bars. 2. **Disjoint Set Union (DSU / Union-Find):** Tracks elements partitioned into disjoint subsets. With two critical optimizations?Path Compression (flattening tree during `find()`) and Union by Rank/Size (attaching smaller tree under deeper root)?operations run in amortized $O(\alpha(n))$ time, where α is the extremely slow-growing inverse Ackermann function (effectively ≤ 4 for all realistic inputs). 3. **Segment Tree:** Complete binary tree storing aggregated range metrics (sum, min, max) over array segments. Allows both arbitrary range queries and point updates in strictly $O(\log n)$ time.

· 2. Key Formulas, Rules & Complexity Classes

1. **DSU Inverse Ackermann Bound:** $$\text{Time per operation} = O(\alpha(n)) \approx O(1)$$ 2. **Segment Tree Properties:** For array of size n, array-backed tree size $\le 4n$. $$\text{Build Time} = O(n), \quad \text{Range Query} = O(\log n), \quad \text{Point Update} = O(\log n)$$

· 3. Step-by-Step Solved Examples

Example 9.1: Trie (Prefix Tree) Implementation

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Microsoft`
- **Problem Statement:** Implement a Trie with insert, search, and startsWith methods.

Step-by-Step Walkthrough & Logic:

- Step 1: Define `TrieNode` having a dictionary of children and a boolean flag `is_end_of_word`.
- Step 2: `insert(word)`: Traverse character by character from root, creating child nodes when missing, and mark final node as `is_end_of_word = True`.

- Step 3: `search(word)`: Traverse characters; if path broken return False; else return `curr.is_end_of_word`.
- Step 4: `starts_with(prefix)`: Traverse characters; if path broken return False; else return True.

Complete Solved Code / Calculation:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        curr = self.root
        for ch in word:
            if ch not in curr.children:
                curr.children[ch] = TrieNode()
            curr = curr.children[ch]
        curr.is_end = True

    def search(self, word: str) -> bool:
        curr = self.root
        for ch in word:
            if ch not in curr.children:
                return False
            curr = curr.children[ch]
        return curr.is_end

    def starts_with(self, prefix: str) -> bool:
        curr = self.root
        for ch in prefix:
            if ch not in curr.children:
                return False
            curr = curr.children[ch]
        return True
```

- **Line-by-Line Breakdown & Complexity Analysis:** All operations take strictly $O(L)$ time where L is the string length. Space is bounded by total characters stored across nodes.

· 4. Practice Questions with Answers & Explanations

Practice Question 9.1

Question: What is the amortized complexity of m DSU operations on n elements with path compression and union by rank?

Options:

- A) $O(m \log n)$
- B) $O(m \alpha(n))$
- C) $O(m n)$
- D) $O(m \log^2 n)$

Answer: **B) $O(m \alpha(n))$** **Detailed Solution & Reasoning:** Robert Tarjan proved that combining union by rank and path compression achieves $O(m \alpha(n))$ across m operations.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 9.1 (Amazon)

- **Interview Question:** _When would you choose a Fenwick Tree (Binary Indexed Tree) over a Segment Tree?_

- **Model High-Scoring Answer:** A Fenwick Tree is simpler to implement (less than 15 lines of code) and uses strictly $O(n)$ space compared to $4n$ memory in Segment Trees. It is optimal for cumulative prefix sum queries and point updates. However, Segment Trees are required when range queries involve non-invertible operations (such as Range Minimum Query or Range GCD) or when range updates require Lazy Propagation.

- **Interviewer Follow-Up & Pro Tip:** ? Mentioning the bitwise arithmetic `x & (-x)` for Fenwick Tree index advancement shows deep familiarity.

CHAPTER 10

Solved Hard Placement DSA Problems & Interview Drills

Key Principle / Exam Takeaway: Master end-to-end solutions to top corporate interview benchmarks: Trapping Rain Water, Merge K Sorted Lists, and Word Ladder.

Core Curriculum Topics Covered in this Chapter:

- Trapping Rain Water: Two-pointer optimal O(n) solution
- Merge K Sorted Lists: Min-Heap optimal O(N log k) solution
- Word Ladder: Bidirectional BFS shortest transformation sequence
- Placement coding interview rubrics, communication strategies, and edge testing

· 1. Core Concepts & In-Depth Explanation

This capstone chapter synthesizes multi-structure algorithms on flagship corporate placement problems frequently asked in final coding rounds at FAANG, Tier 1 product firms, and premium IT services (TCS Digital, Infosys Power Programmer).

In high-stakes technical interviews, arriving at the correct answer is only 50% of the evaluation rubric: 1. **Clarifying Requirements & Constraints:** Confirming input boundaries, null/empty cases, negative values, and memory limitations. 2. **Brute Force Baseline:** Articulating the naive solution and identifying computational bottlenecks. 3. **Optimal Design & Intuition:** Communicating the algorithmic trade-off clearly before touching code. 4. **Clean, Bug-Free Code:** Writing modular, well-named, production-quality code. 5. **Dry Run & Edge Testing:** Tracing through edge cases (single-element, duplicate elements, empty arrays) to prove correctness.

· 2. Key Formulas, Rules & Complexity Classes

1. **Trapping Rain Water Invariant:** $$\text{WaterAt}(i) = \max(0, \min(\text{MaxLeft}[i], \text{MaxRight}[i]) - height[i])$$ 2. **Merge K Sorted Lists Complexity:** $$N = \text{total nodes across all } k \text{ lists} \implies \text{Time} = O(N \log k), \quad \text{Space} = O(k)$$

· 3. Step-by-Step Solved Examples

Example 10.1: Trapping Rain Water (Two Pointers)

- **Difficulty / Target Companies:** `Hard | Google, Amazon, Microsoft, Uber`
- **Problem Statement:** Given n non-negative integers representing an elevation map where width of each bar is 1, compute how much water it can trap after raining.

Step-by-Step Walkthrough & Logic:

- Step 1: Set `left = 0`, `right = n - 1`, `left_max = 0`, `right_max = 0`, `water = 0`.
- Step 2: While `left < right`: compare `height[left]` and `height[right]`.
- Step 3: If `height[left] < height[right]`: if `height[left] >= left_max`, update `left_max = height[left]`; else add `left_max - height[left]` to `water`. Increment `left`.

- Step 4: Else: if `height[right] >= right_max`, update `right_max = height[right]`; else add `right_max - height[right]` to `water`. Decrement `right`.
- Step 5: Return `water`.

Complete Solved Code / Calculation:

```
def trap(height):
    if not height:
        return 0
    left, right = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1
    return water
```

- **Line-by-Line Breakdown & Complexity Analysis:** Executes in strictly $O(n)$ time with $O(1)$ auxiliary memory. The two pointers maintain dynamic boundaries without needing $O(n)$ prefix arrays.

Example 10.2: Merge K Sorted Lists (Min-Heap)

- **Difficulty / Target Companies:** `Hard | Amazon, Facebook, Google`
- **Problem Statement:** Merge k sorted linked lists and return it as one sorted list.

Step-by-Step Walkthrough & Logic:

- Step 1: Create a Min-Heap and insert the head node of all k non-empty lists.
- Step 2: Maintain a dummy sentinel node and a pointer `tail`.
- Step 3: While heap is not empty: pop node with minimum value and attach to `tail.next`. Advance `tail`.
- Step 4: If popped node has a `.next`, push that next node into the min-heap.
- Step 5: Return `dummy.next`.

Complete Solved Code / Calculation:

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            # Tuple contains (val, index, node) to break ties without comparing ListNode object
            heapq.heappush(heap, (node.val, i, node))
```

```
    dummy = ListNode(0)
    curr = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))

    return dummy.next
```

- **Line-by-Line Breakdown & Complexity Analysis:** The heap stores at most k nodes at any point. Popping and pushing takes $O(\log k)$. With N total elements across all lists, running time is $O(N \log k)$ and auxiliary space is $O(k)$.

· 4. Practice Questions with Answers & Explanations

Practice Question 10.1

Question: What is the optimal time complexity to solve the Median of Two Sorted Arrays problem?

Options:

- A) $O(m + n)$
- B) $O(\log(m + n))$
- C) $O(\min(m, n))$
- D) $O((m + n) \log(m + n))$

Answer: **B) $O(\log(m + n))$** **Detailed Solution & Reasoning:** Binary searching over the partition point of the smaller array divides the combined elements into two equal halves in $O(\log(\min(m, n)))$ time.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 10.1 (Google)

- **Interview Question:** _In a coding interview, you write working code that passes all sample tests. What should you do before telling the interviewer you are finished?_

- **Model High-Scoring Answer:** Never immediately say you are done. 1) Manually trace the code step-by-step with a concrete sample input to verify variable states. 2) Proactively test boundary edge cases: empty input, single element, all duplicates, negative numbers, and extreme values. 3) Check for memory leaks or integer overflows. 4) State the final time and auxiliary space complexity clearly. This structured diligence signals senior software engineering habits.

- **Interviewer Follow-Up & Pro Tip:** ? Interviewers frequently wait to see if candidates catch their own bugs during manual dry runs.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.